



38th International Conference on VLSI Design & 24th International Conference on Embedded System



SILICON MEETS AI : Sustainable Innovation In Accelerated Computing, Secure Connectivity, And Intelligent Mobility
4th - 8th January 2025 | Bengaluru, India

Primer on Data in Quantum Machine Learning

Aviral Shrivastava, Vinayak Sharma,
Arizona State University

Set the Expectations

We hope you know

- Some prior experience with machine learning. **This would include**
 - ✓ Linear & Logistic Regression
 - ✓ Kernel Classifiers/SVMs
 - ✓ Basics of Deep Neural Networks
- Some background in quantum computing . **This would include:**
 - ✓ Qubits and Single Qubit gates
 - ✓ Measurement in Quantum Computing
 - ✓ Multi-Qubit gates and Entanglement

What do we expect you to learn

- Understand what machine learning looks like on Near Intermediate Scale Quantum (**NISQ**) devices.
- Explore the most popular families of Quantum Machine Learning (QML) models
- Understand the differences between these model families and how they relate to each other.

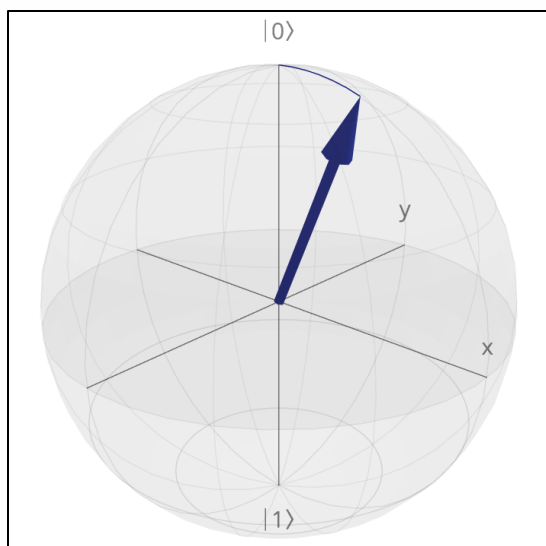
How do we expect you to learn

- **Math** – We will look at some of the mathematical differences and implications of the models
- **Code** – We will be going through code implementations of the model families in order to better understand the implementations. All code will be in PennyLane

Basics of Quantum Computing

Quantum State

The state of a quantum system is defined in terms qubits. Each qubit has an associated state which is a point on the surface of a hypersphere (*Bloch Sphere*)

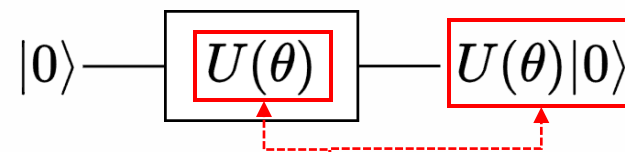


$$|\psi\rangle = \cos(0.2\pi) |0\rangle + \sin(0.2\pi) |1\rangle$$

Quantum State Information

Quantum Computation

1. All quantum computation is carried out on the quantum state of the qubits.



Gates act on the quantum states (i.e.) the state vectors

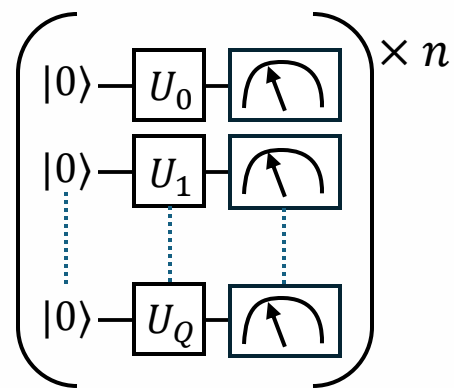
2. The postulates of Quantum Mechanics define the types of computation we can do.

Quantum Computation is bounded by 3 main rules:

No Cloning theorem	State Collapse	Unitary Constraint
The no cloning theorem limits our ability to make state copies	State Collapse prevents us from observing intermediate computation steps	The unitary constraints of reversibility and normality limit compute operations we can undertake

State Measurement

General Idea: While the information a single run of the quantum circuit provides is limited, if we know the gates applied, we can run a circuit multiple times (often called shots) to extract more useful statistical information.



Running this 'Q' qubit circuit ' n ' times provides us with ' n ' eigen values. We can represent this as a row (M) vector of shape (n ,) of the form :-

$$M = \{m_i \mid m_i \in \{-1, 1\}^{\otimes Q} \text{ and } i \in [0, n]\}$$

From this set of readings, we can now compute statistically useful properties.

Statistical Readings from Measurements

Counts

- The number of times each eigen value occurs.
- Ex: $-1, \dots, n \text{ times} = t$ occurrences

Probability

- The probability of any eigenvalue.
- Ex:
 $P(-1, \dots, n \text{ times}) = p_n$

Expectation Value

- The expected value of the measurement operation on that state.
- Ex: $E[M] = \sum_{i \in M} i \cdot P(i)$

Variance

- The variance in the readouts.
- Ex: $E[M^2] - E[M]^2$



38th International Conference on VLSI Design & 24th International Conference on Embedded System



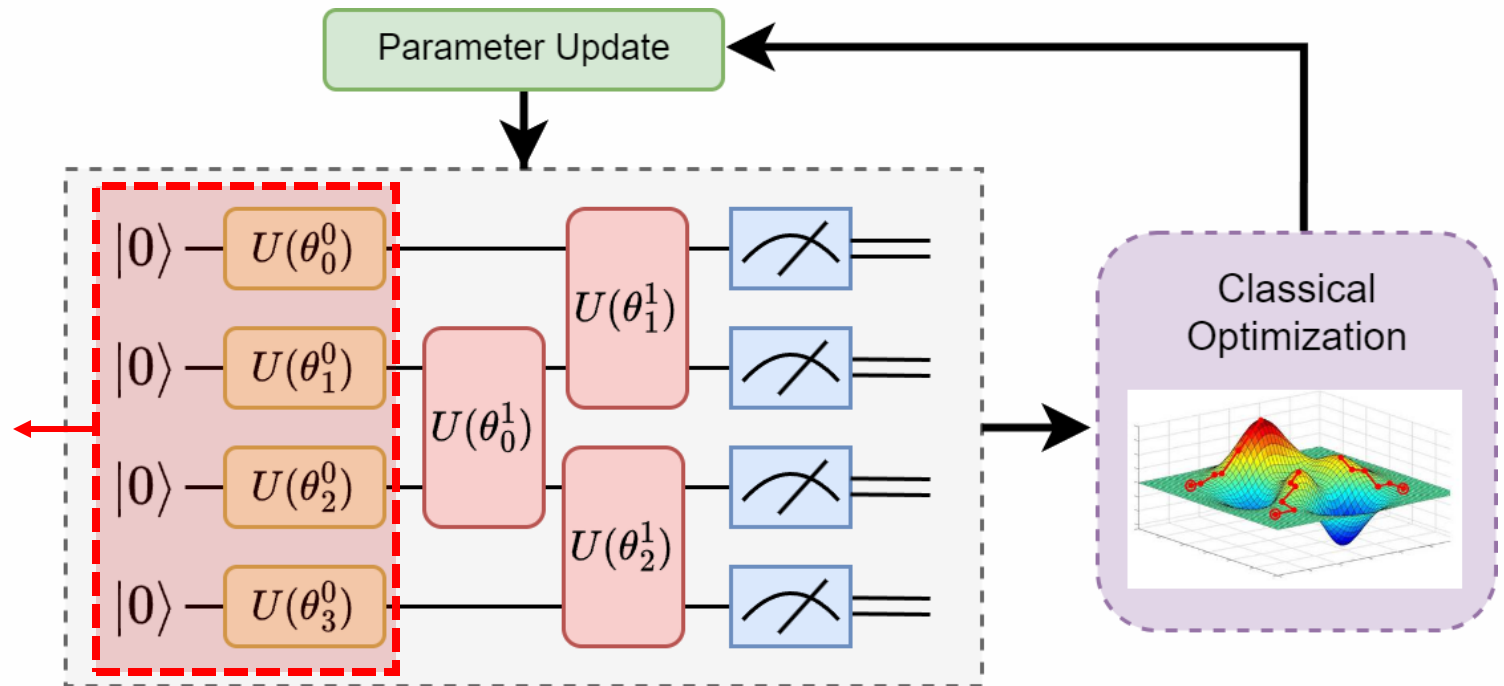
SILICON MEETS AI : Sustainable Innovation In Accelerated Computing, Secure Connectivity, And Intelligent Mobility
4th - 8th January 2025 | Bengaluru, India

Section 2 - Quantum Machine Learning. QML Overview

Anatomy of a QML Circuit

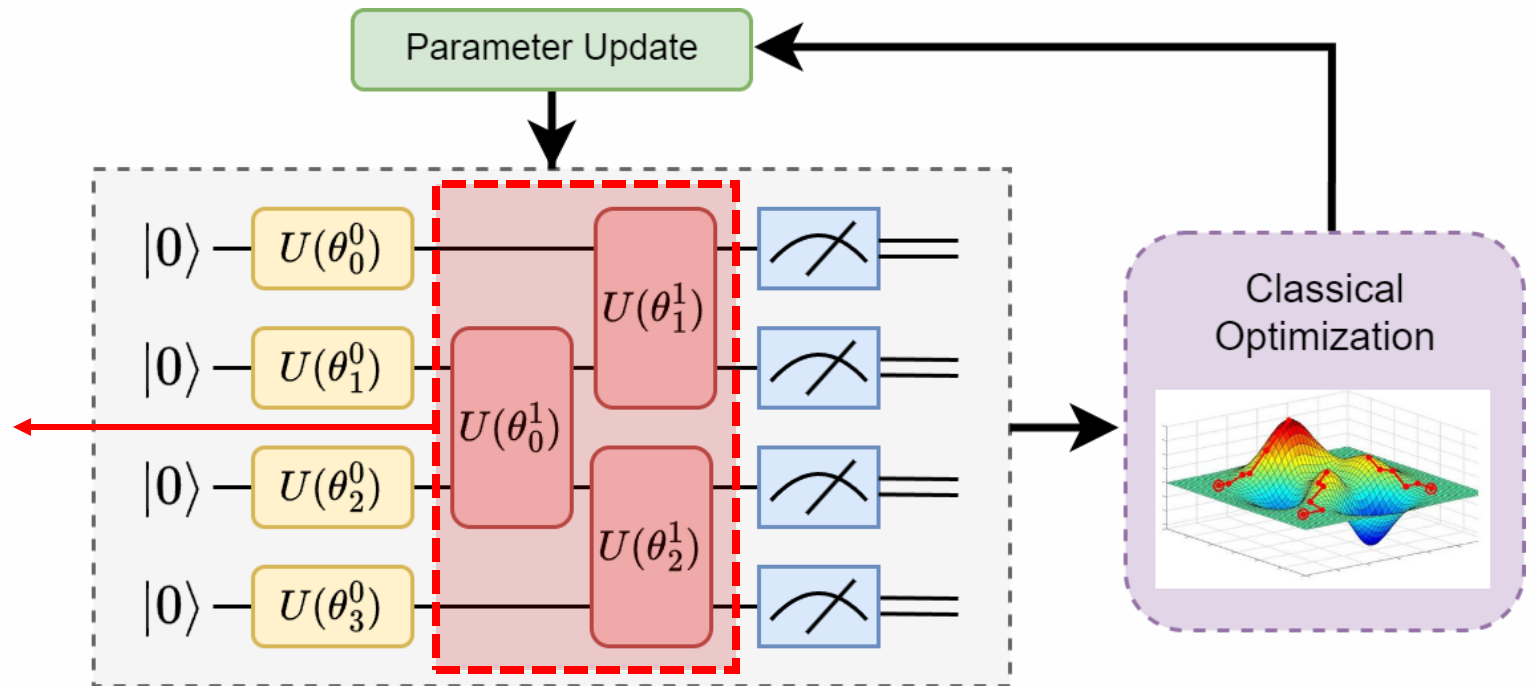
Data Encoding

In order to compute on classical data, we need to encode it into a quantum state. This is done via some parameterized unitary where our data is the parameter.



Anatomy of a QML Circuit

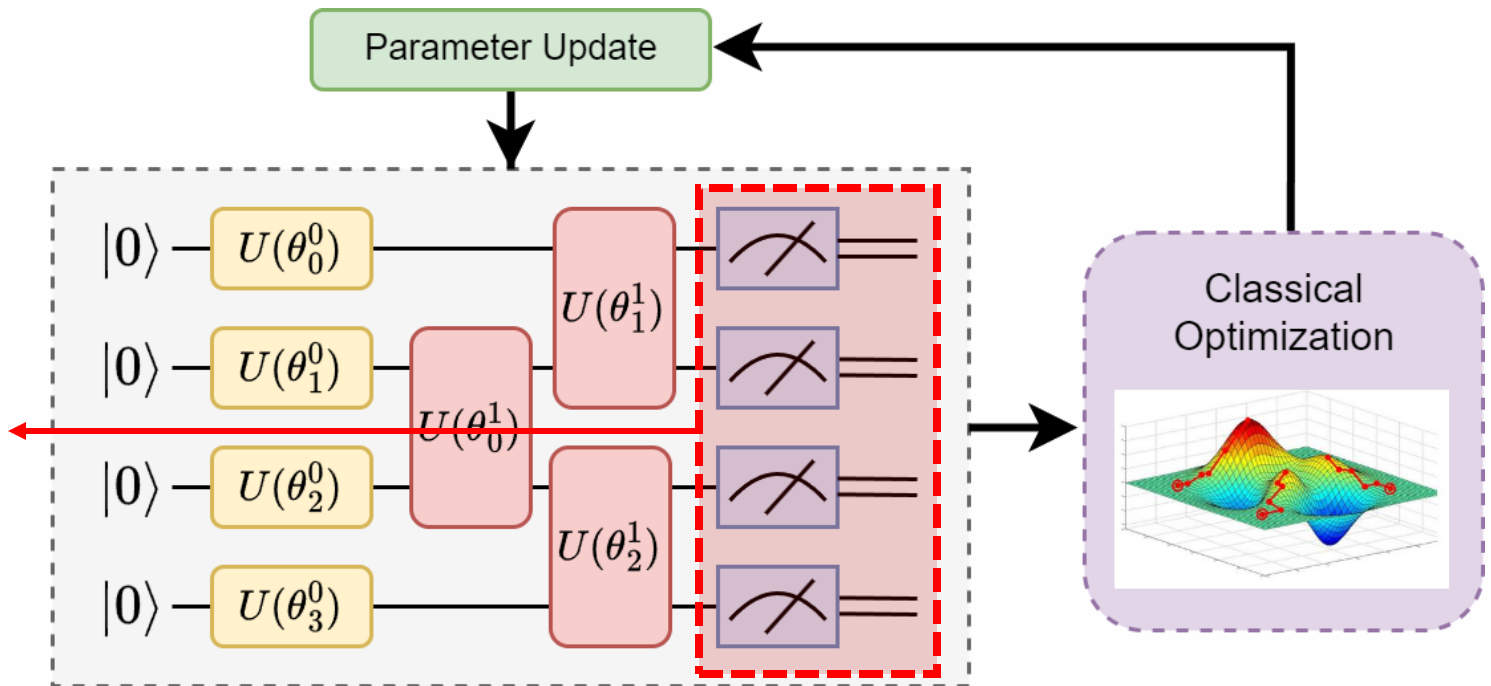
Trainable Quantum Circuit
Also known as “Parameterized Quantum Circuits” (PQC). They normally use rotational gates (R_X, R_Y, R_Z) with trainable rotational angles (θ).



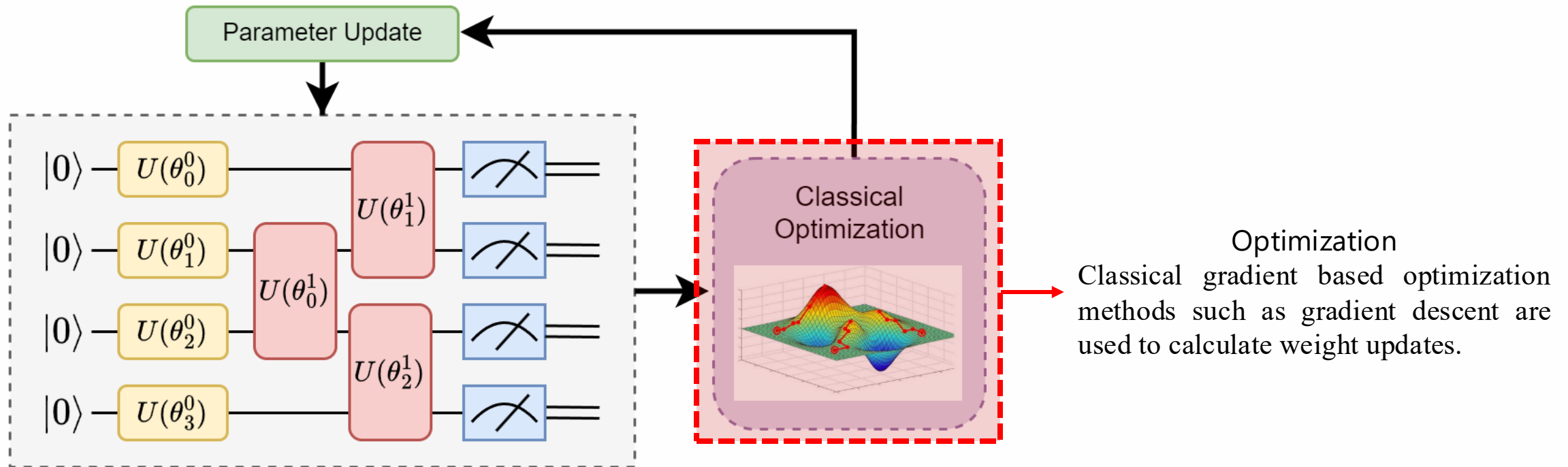
Anatomy of a QML Circuit

Measurement

Every quantum circuit can only output a result of measurement. A single measurement only produces a single basis state (generally $|0\rangle$ or $|1\rangle$). We can use multiple measurements to extract statistical values such as probability of a basis state or expectation values. These measurement outputs are incorporated into loss functions.

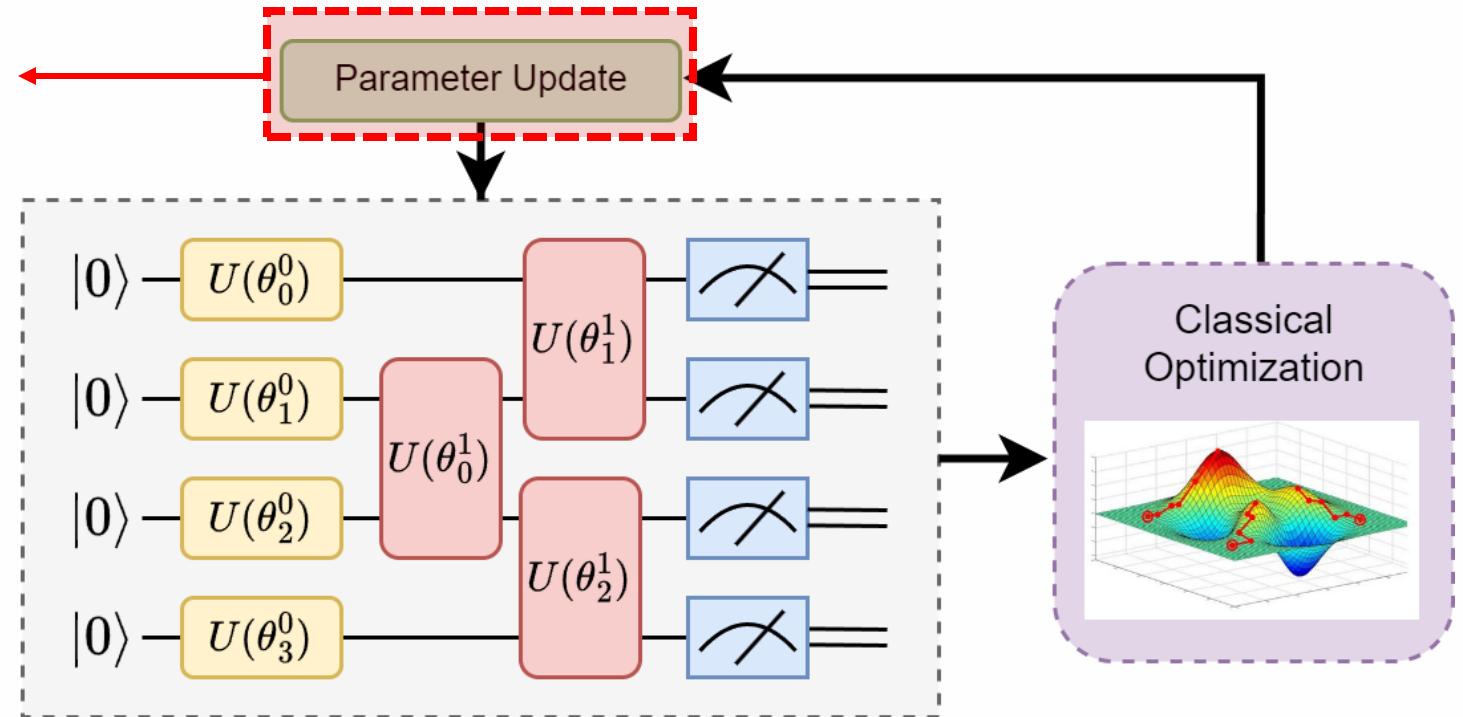


Anatomy of a QML Circuit



Anatomy of a QML Circuit

Parameter Update
The rotational angles are updated based on the calculated gradients.



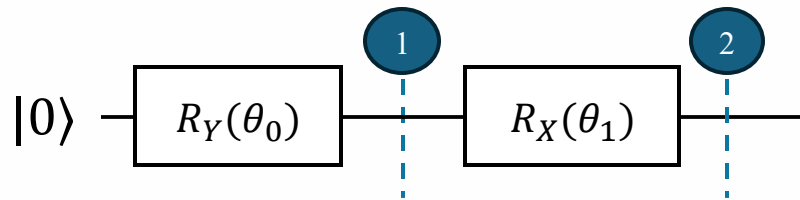
Gradients in Quantum Machine Learning

Classical Backpropagation

As might be apparent, analytical differentiation and more importantly backpropagation relies on caching the results from forward passes.

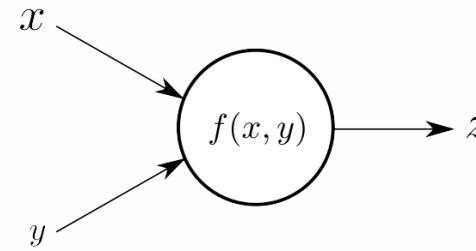
The Problem !!!

Due to the measurement principle, we cannot access the intermediate states of a quantum circuit. This makes it difficult to compute gradients. Consider the circuit below -

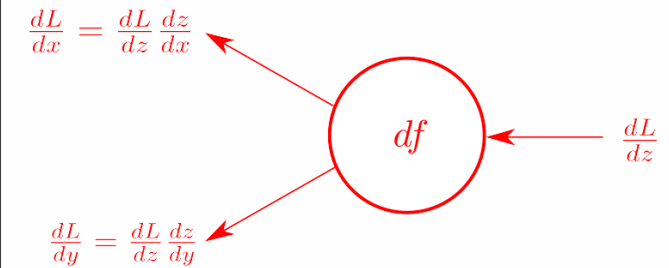


Without access to (1), we cannot compute the derivative for the first gate.

Forwardpass



Backwardpass



However, this is not a problem for simulators... As simulators always have access to the entire state vector, we can compute gradients on them using the analytical method and gradient descent.

Gradients on Real Devices

While backprop works for simulators, we cannot compute gradients on real devices using that method, the use of real quantum devices is limited.

Solution ... Numerical Methods

Unlike analytical methods, numerical methods can compute gradients up to a level of noise using the outputs.

An example of this is the 'finite difference' method which approximates gradients by discretizing the outputs. However, these values are approximate and error prone.

Enter ... Parameter Shift

The parameter shift method leverages the special properties of quantum gates to produce 'exact gradients'.

Parameter-Shift

Recall that,

Our gates $U(\theta)$ can w.l.o.g be written as a composition of Pauli-rotational gates:

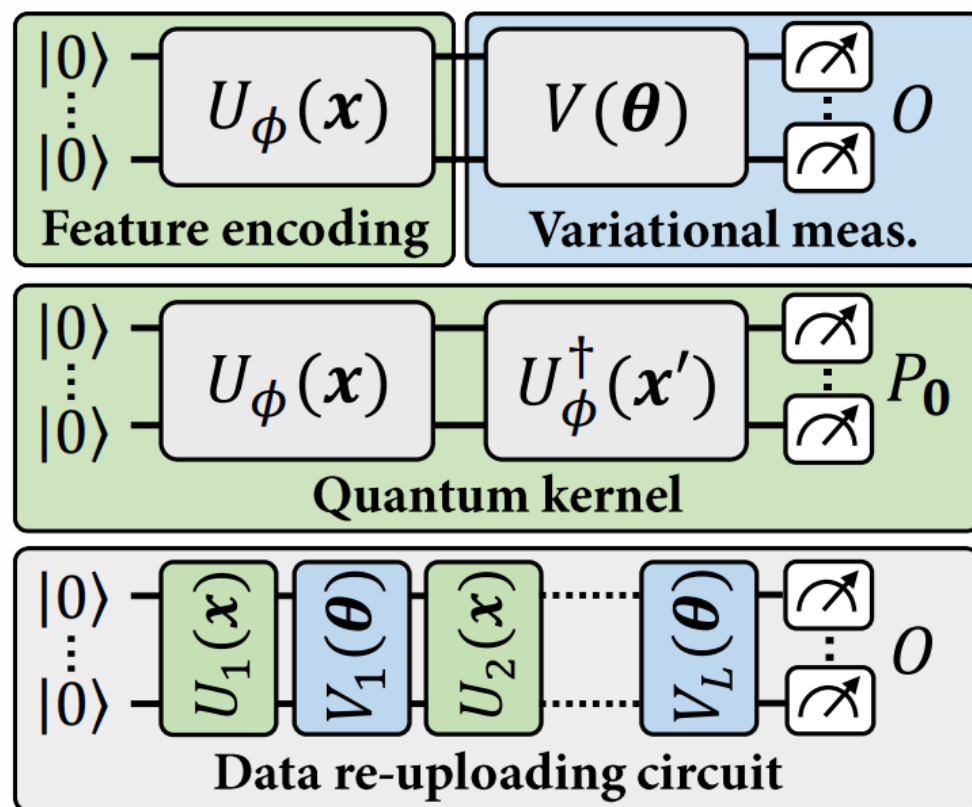
$$\text{i.e., } U(\theta) = R_X(\theta_0).R_Y(\theta_1).R_Z(\theta_2)$$

Rotational gates are function of sin and cosine functions, **whose derivatives are just function shifted versions of themselves**. This allows us to efficiently compute derivatives by running the forward pass with shifted parameters.

Example: $\nabla_{\theta} \sin(\theta) = \cos(\theta) = \sin\left(\frac{\pi}{2} - \theta\right) = \sin\left(\theta + \frac{\pi}{2}\right)$ (original function shifted by a parameter $(\frac{\pi}{2})$)

$$R_Y(\theta) = \begin{bmatrix} \cos(\frac{\theta}{2}) & -\sin(\frac{\theta}{2}) \\ \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{bmatrix} \quad \frac{d}{d\theta} R_Y(\theta) = \begin{bmatrix} -\sin(\frac{\theta}{2}) & -\cos(\frac{\theta}{2}) \\ \cos(\frac{\theta}{2}) & -\sin(\frac{\theta}{2}) \end{bmatrix}$$

Families of QML Models



Source: [1]

Explicit Model

An **explicit quantum model**, where the label of a data point x is specified by the expectation value of a variational measurement on its associated quantum feature state $\rho(x)$

Implicit Model

The quantum kernel associated to these quantum feature states. The expectation value of the projection $P_0 = |0\rangle\langle 0|$ corresponds to the inner product between $\rho(x)$ and $\rho(x')$. An **implicit quantum model** is defined by a linear combination of such inner products, for x an input point and x' training data points.

Data Reuploading Model

A data re-uploading model, interlaying data encoding and variational unitaries before a final measurement. It draws inspiration from classical neural networks



38th International Conference on VLSI Design & 24th International Conference on Embedded System



SILICON MEETS AI : Sustainable Innovation In Accelerated Computing, Secure Connectivity, And Intelligent Mobility
4th - 8th January 2025 | Bengaluru, India

Section 3 - Explicit Models.. Looking at the Details

Data Encoding

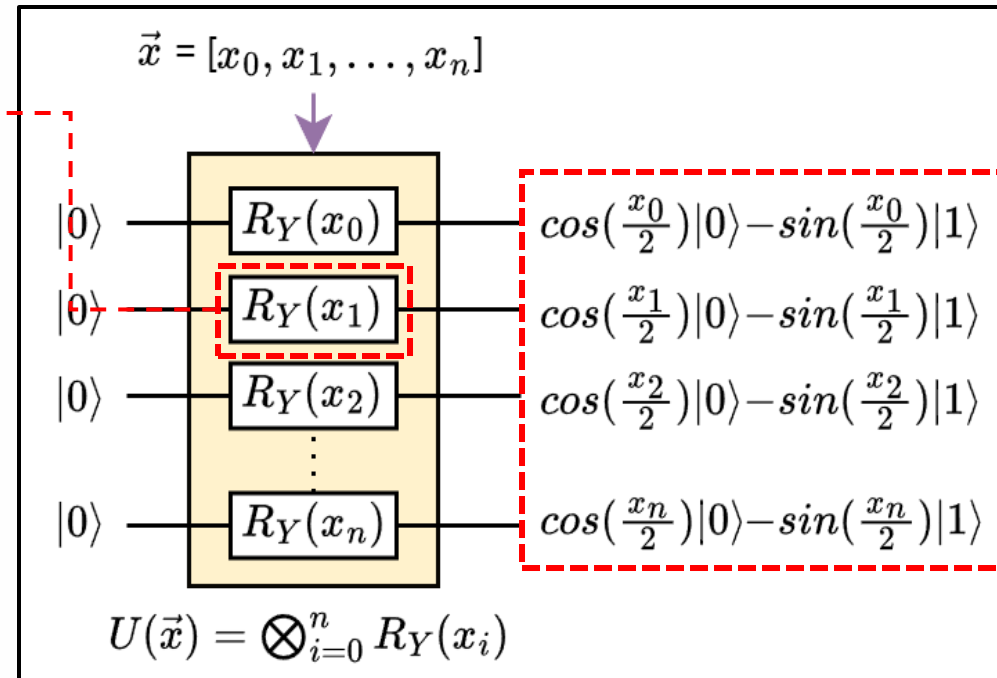
$U(\vec{x})|0\rangle^{\otimes n}$ Encoded State

The R_Y Gate creates the state seen above

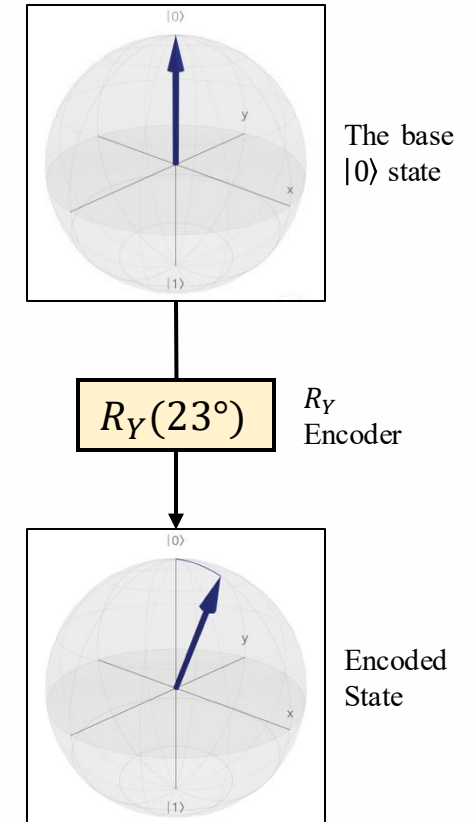
$$R_Y(\theta) = \begin{pmatrix} \cos(\frac{\theta}{2}) & -\sin(\frac{\theta}{2}) \\ \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{pmatrix}$$

R_Y Gate matrix formula. As evident, the R_Y gate is parameterized by the angle θ .

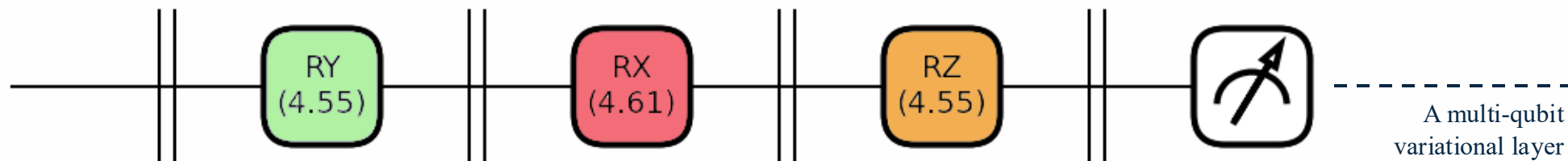
- During data encoding θ are the classical scalars to be encoded.
- During data processing, they are trainable parameters.



Data Encoding Circuit. The data encoding circuit has 0 trainable parameters. The circuit encodes classical data onto the state of the quantum computer. The most common method is using rotational gates.

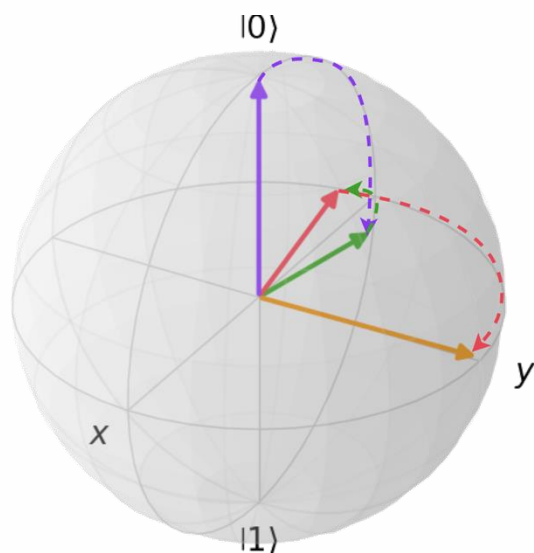


Data Processing



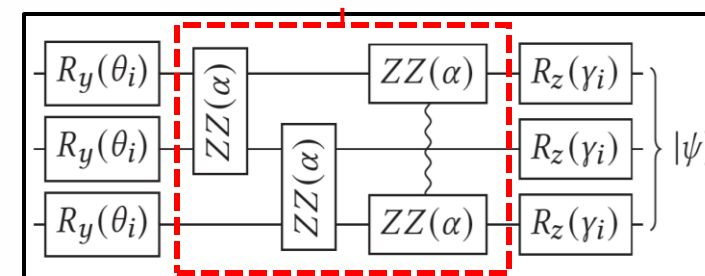
A multi-qubit variational layer generally includes entanglement and multiple sets of rotational gates.

Entanglement creates intractable feature maps, which help with training.



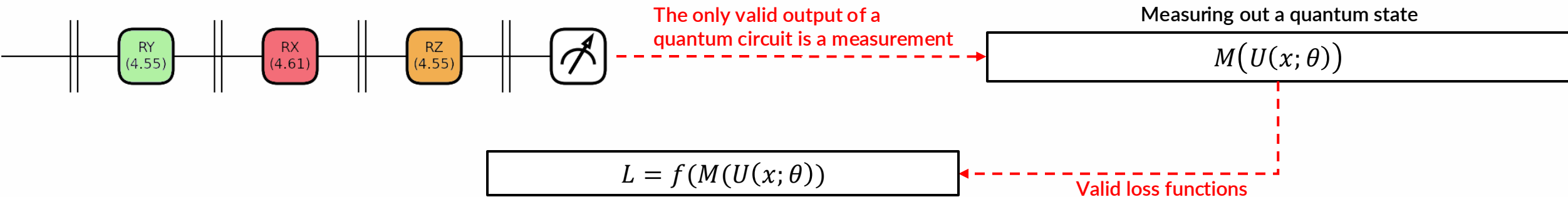
Rotational Gates cause our state vector to rotate along the Bloch sphere.

This creates functional mappings on our encoded data.



Source:[1] | Parameterized Quantum Circuit using R_y and R_z Gates. The circuit also uses a $ZZ(\theta)$ Gate to create entanglement across multiple qubits.

Circuit Output and Loss Functions



Statement: Any valid loss function for a hybrid model must be a function of the circuit measurement.

NEW PROBLEM !
What about the gradients then?

This is a direct result of the quantum state collapse property which **limits us to only knowing the results of the measurements of quantum circuits.**

Therefore, any valid quantum circuit must yield a measurement. Hence, the loss must also be based on measurements.

Explicit Models as Hyperplanes – Projective Measurement I

Projection Valued Measures

Definition PVM Measurement in Quantum Computing can be thought of as a 'Projection' from our 'Quantum Feature Space' onto a 'Observable'

Let's break this down !!!

Quantum Feature Space

Definition: A Quantum Feature Space can be formally defined as the surface of a hyper sphere in a 2^n -dimensional Hilbert Space.

Where, n = number of qubits.

A Hilbert Space is a vector space that is inner product complete. (i.e. $\langle u, v \rangle \in H \forall u, v \in H$)

Simplifying: We can think of it as a coordinate plane with 2^n axes. The additional property is that our points need to satisfy the equation of a unit sphere in this space.

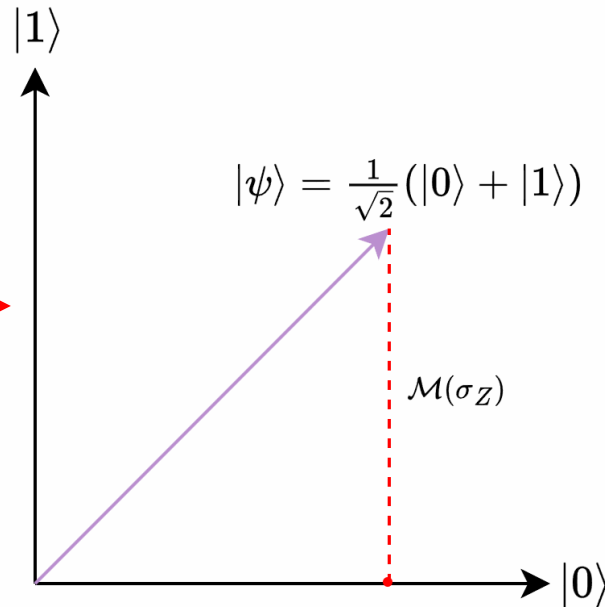
Explicit Models as Hyperplanes – Projective Measurement II

Definition PVM Measurement in Quantum Computing can be thought of as a **Projection** from our **Quantum Feature Space** onto a **Observable**

Measurement as Projection

Definition: Measurement of a quantum system can be defined as projecting a state (ψ) into some operator (O).

$$M_O = \langle \psi | O | \psi \rangle$$



Quantum Observables/ Measurement Operators

Definition: An observable is any quantity that can be measured. In quantum mechanics, these are normally **Quantum Operators**.

An operator is any quantum gate that is **self-adjoint** (i.e. is its own inverse) and has **real eigen values**.

Computational Basis

$$\sigma_Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

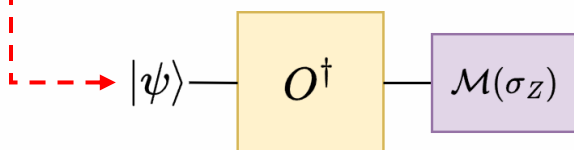
Pauli-Z Operator

The “Computational Basis” is when we measure against the Pauli-Z (σ_Z) Operator.

Measurement in Arbitrary Basis

Measuring in an arbitrary basis.

We can perform measurement of a state w.r.t to any arbitrary operator or basis by applying the inverse of the operator to our state and then measuring in the standard basis.



Proof:

Statement: The measurement operation on our evolved state $O^\dagger|\psi\rangle$ w.r.t $\sigma_Z \equiv |\langle\psi|O|0\rangle|^2$ --(1). Assuming, that the operator O produces the state $|\phi\rangle$ when applied to $|0\rangle$ (i.e. $|\psi\rangle = O|0\rangle$) our previous equation (1) simplifies to $|\langle\psi|\phi\rangle|^2$.

Proof:

Let our initial state be $|\psi\rangle$, Applying the operator $O^\dagger$ on our state would result in our new state $|\Omega\rangle = O^\dagger|\psi\rangle$.

Clearly ,

$$\langle\Omega| = (|\Omega\rangle)^\dagger = (O^\dagger|\psi\rangle)^\dagger = \langle\psi|O$$

Applying our measurement operation w.r.t our σ_Z projector $|0\rangle\langle 0|$,

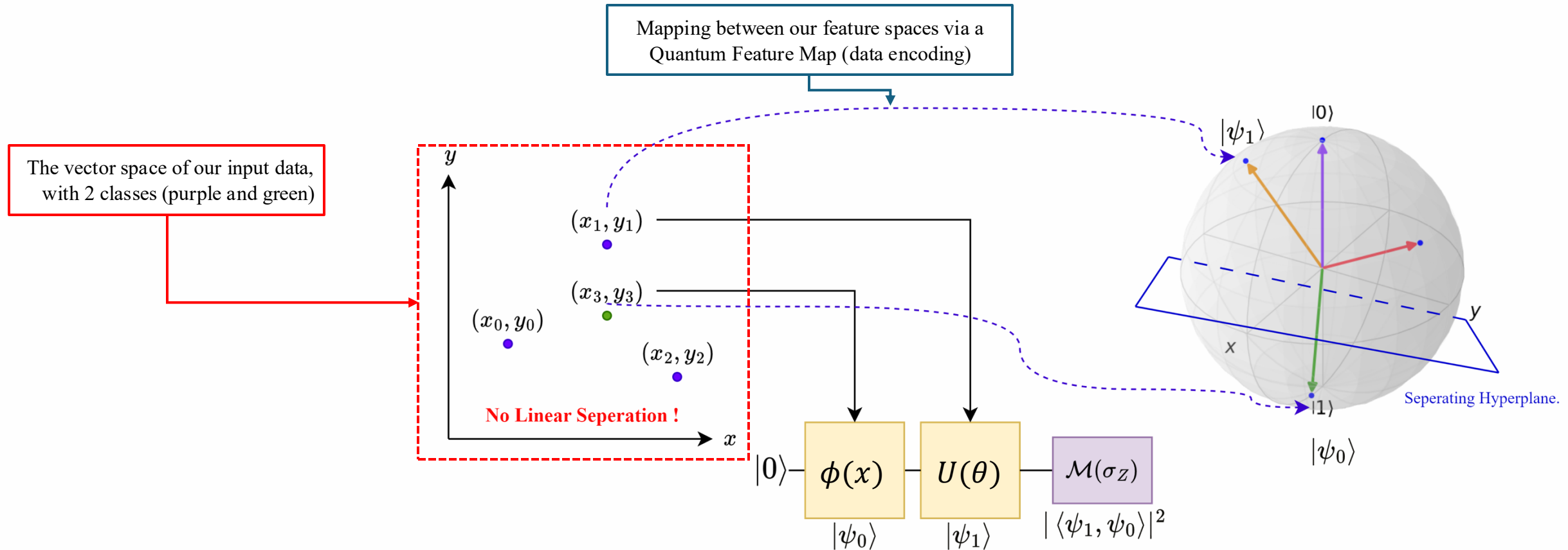
$$\langle\Omega|0\rangle\langle 0|\Omega\rangle = \langle\psi|O|0\rangle\langle 0|O^\dagger|\psi\rangle \text{ --(3)}$$

As the inner product here produces a real scalar, they are unaffected by applying the conjugate operator. (i.e. $\langle a|b\rangle = \langle b|a\rangle$)

$$\therefore \text{Eq. 3} = \langle\psi|O|0\rangle(\langle 0|O^\dagger|\psi\rangle)^\dagger = \langle\psi|O|0\rangle\langle\psi|O|0\rangle = |\langle\psi|O|0\rangle|^2 = |\langle\psi|\phi\rangle|^2$$

Note: This proof can be generalized to n-qubit states by assuming our operator and initial states to be multi-qubit states and replacing the $|0\rangle\langle 0|$ projector with $|0\rangle^n\langle 0|^n$

Explicit Models as Hyperplanes

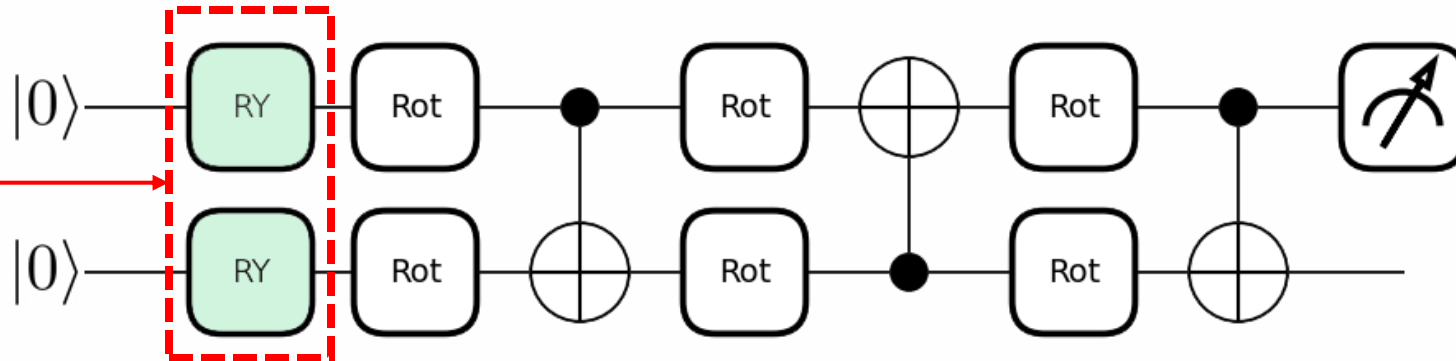


Explicit Model in Code

Data Encoding

The data encoding block uses R_Y Gates to encode our target classical data into the amplitude of our quantum state.

```
@qml.qnode(dev, interface='tf')
def explicit_model(input, weights):
    qml.RY(input[0], wires=0)
    qml.RY(input[1], wires=1)
    for layer in range(weights.shape[0]):
        for wire in range(2):
            qml.Rot(*weights[layer][wire], wires=wire)
        if(layer%2==0):
            qml.CNOT([0, 1])
        else:
            qml.CNOT([1, 0])
    return qml.expval(qml.PauliZ(0))
```



$$\rightarrow E(\psi) = \langle \psi | Z | \psi \rangle$$

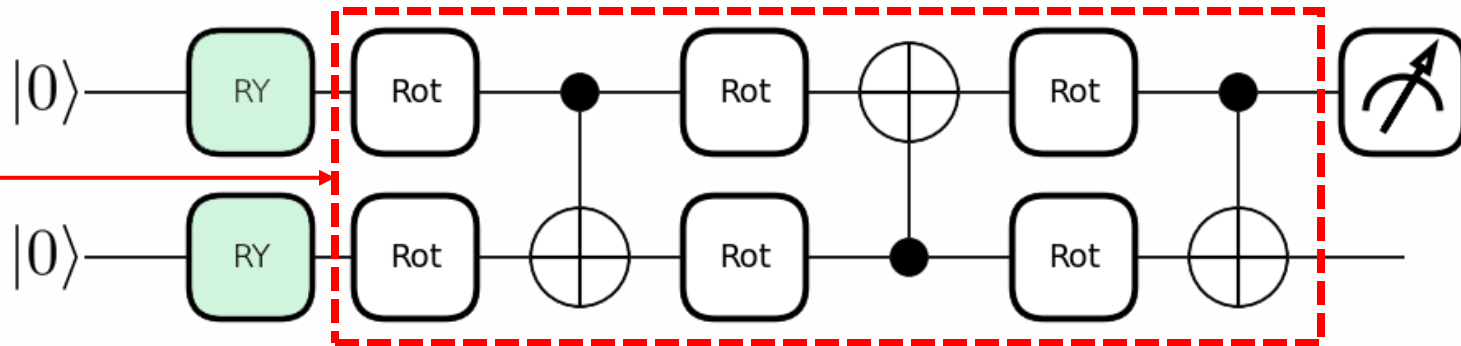
$$|\psi\rangle = U(\theta_{L1}) \cdot U(\theta_{L0}) \cdot (R_Y(x^0) \otimes R_Y(x^1)) |0\rangle$$

Explicit Model in Code

Variational Circuit

The variational circuits consists of parameterized $Rot(\theta_X, \theta_Y, \theta_Z)$ which are equivalent to $R_X(\theta_X) \cdot R_Y(\theta_Y) \cdot R_Z(\theta_Z)$. Layers of them are separated by CNOT entanglers.

```
@qml.qnode(dev, interface='tf')
def explicit_model(input, weights):
    qml.RY(input[0], wires=0)
    qml.RY(input[1], wires=1)
    for layer in range(weights.shape[0]):
        for wire in range(2):
            qml.Rot(*weights[layer][wire], wires=wire)
        if(layer%2==0):
            qml.CNOT([0, 1])
        else:
            qml.CNOT([1, 0])
    return qml.expval(qml.PauliZ(0))
```



$$\rightarrow E(\psi) = \langle \psi | Z | \psi \rangle$$

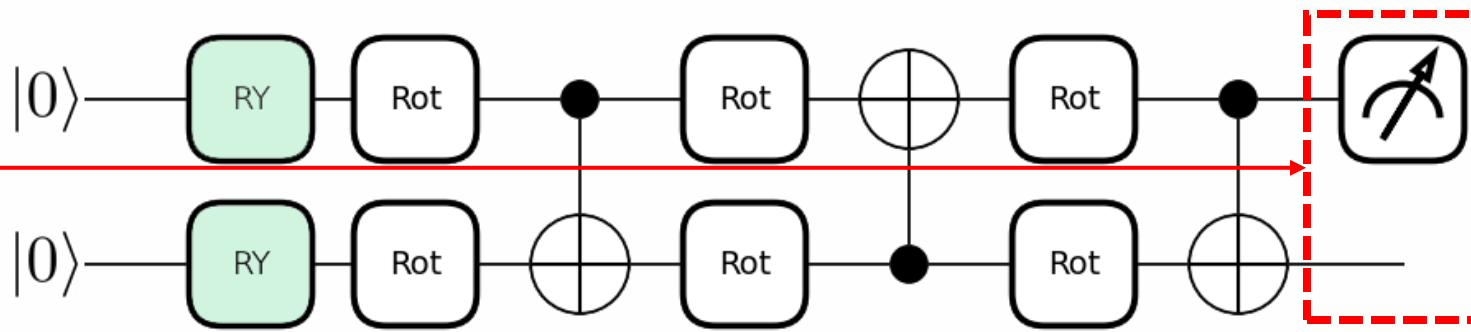
$$|\psi\rangle = U(\theta_{L1}) \cdot U(\theta_{L0}) \cdot (R_Y(x^0) \otimes R_Y(x^1)) |0\rangle$$

Explicit Model in Code

Measurement

Finally, we measure the expectation value w.r.t to the Pauli-Z operator (or computational basis) to be used for our loss function.

```
@qml.qnode(dev, interface='tf')
def explicit_model(input, weights):
    qml.RY(input[0], wires=0)
    qml.RY(input[1], wires=1)
    for layer in range(weights.shape[0]):
        for wire in range(2):
            qml.Rot(*weights[layer][wire], wires=wire)
        if(layer%2==0):
            qml.CNOT([0, 1])
        else:
            qml.CNOT([1, 0])
    return qml.expval(qml.PauliZ(0))
```



$$\rightarrow E(\psi) = \langle \psi | Z | \psi \rangle$$

$$|\psi\rangle = U(\theta_{L1}) \cdot U(\theta_{L0}) \cdot (R_Y(x^0) \otimes R_Y(x^1)) |0\rangle$$

Input/Output Space

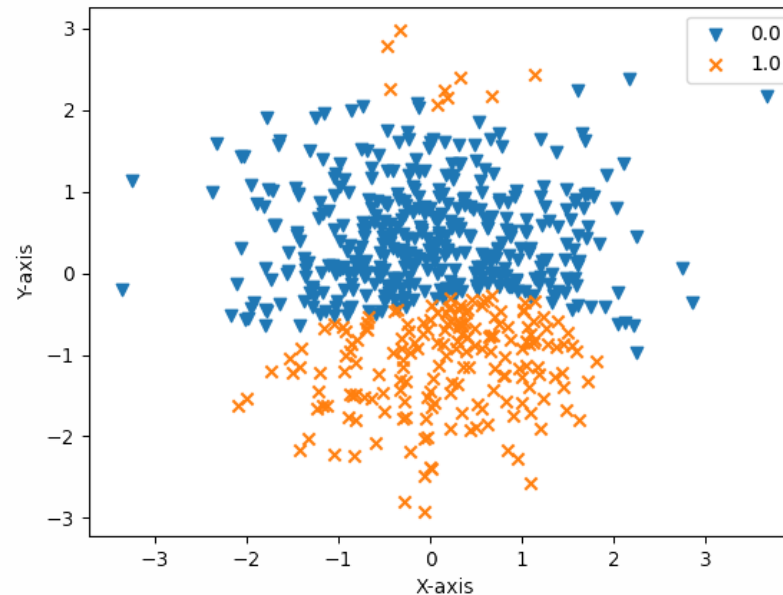
Input/Output Space

In QML, the feature space can also refer to functional space that represents:

$$\mathbb{C}^n \rightarrow \mathbb{R}$$

Where, n = number of input features.

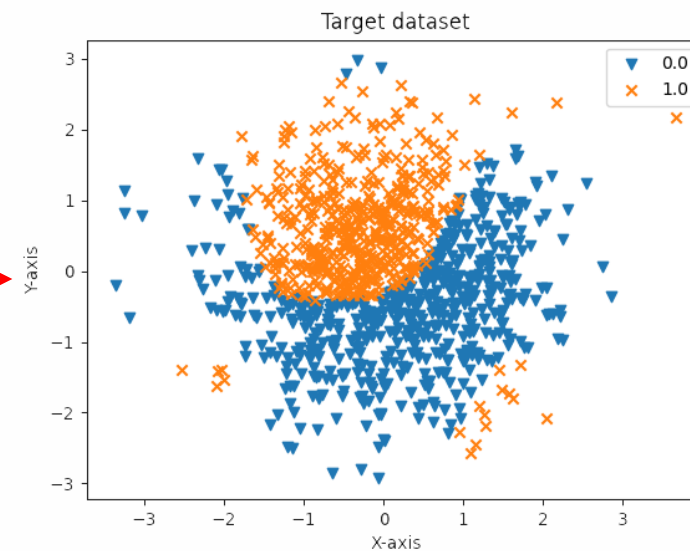
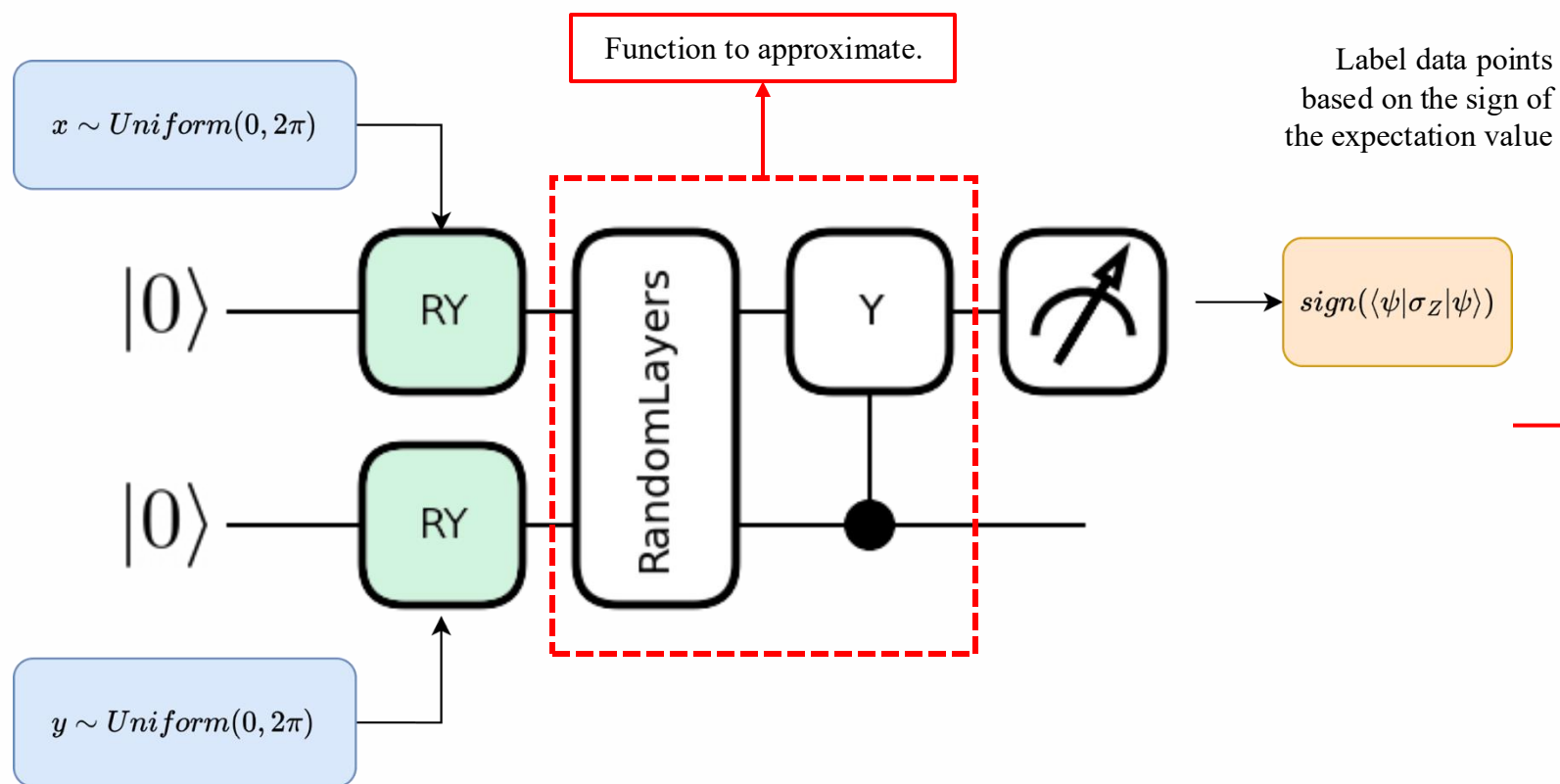
This represents the function space of our inputs to outputs. The **inputs may be complex**, but **measurements always yield real values**.



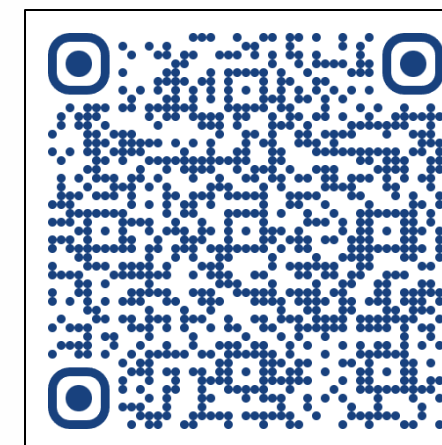
The space on the left is the projection of a 3-D space where the **x,y coordinates correspond to the inputs** and the **z-axis would represent the circuit output**.

Here we project the data onto the X-Y plane and use the np.sign function to label our data. (This would make the $z=0$ our separating hyperplane)

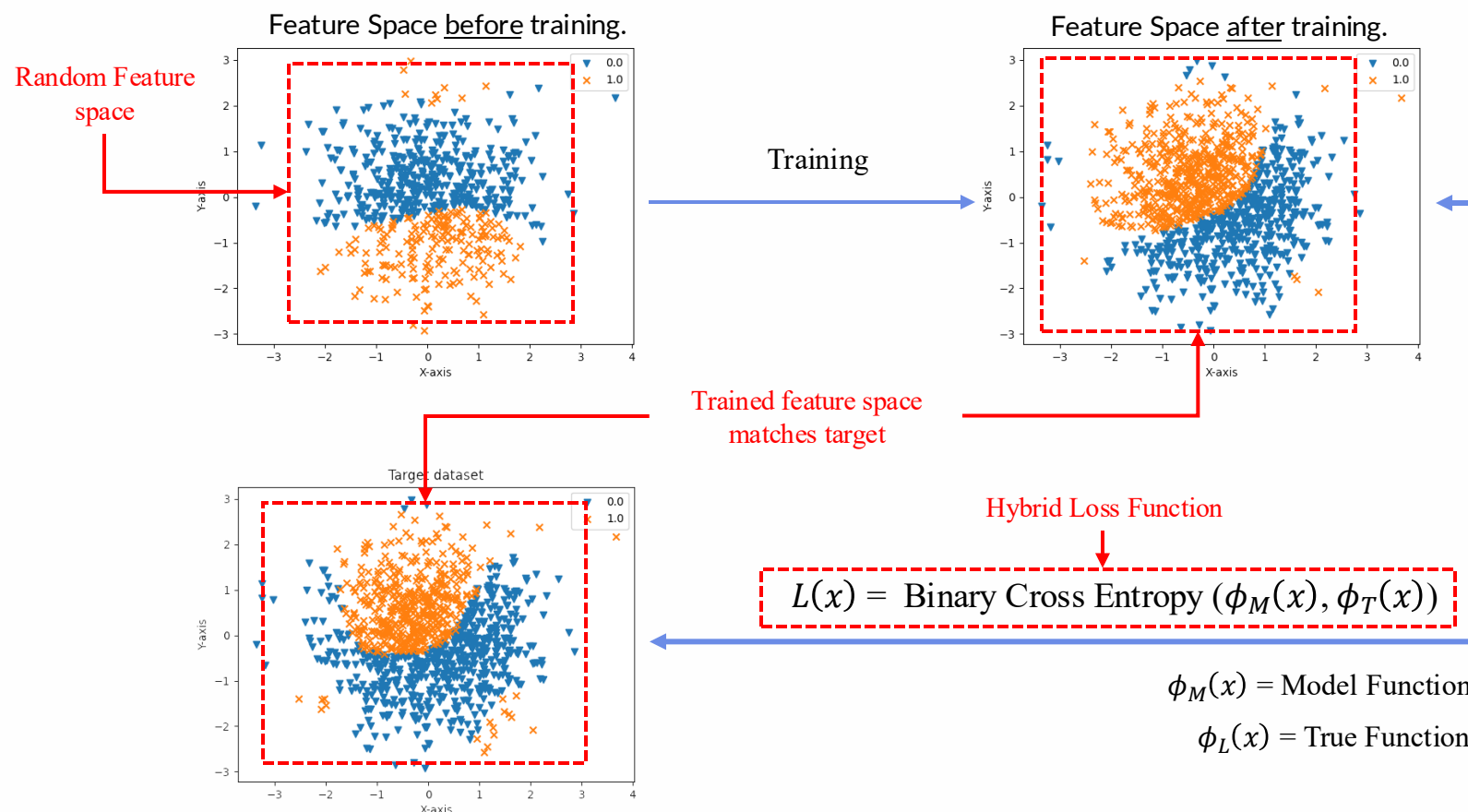
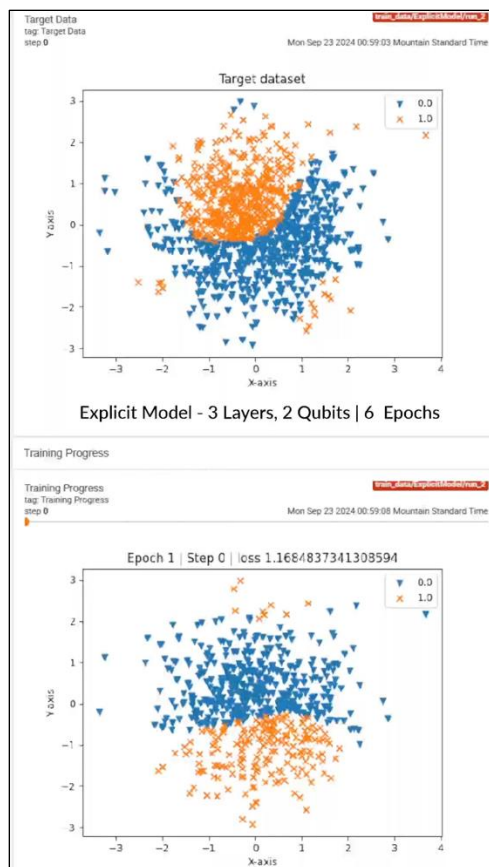
Code Demos



Data Generator

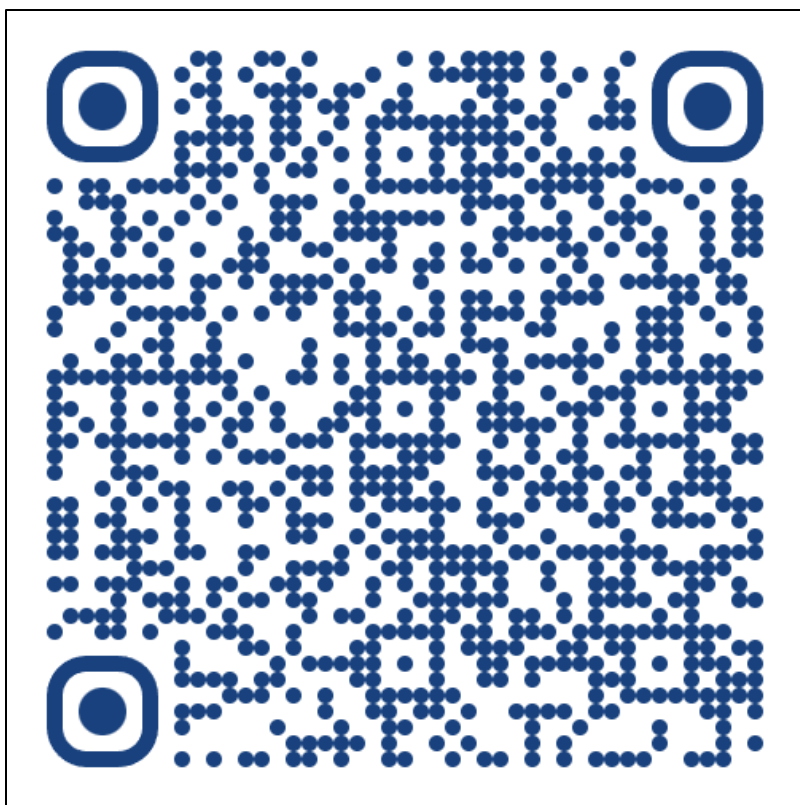


Training Evolution



Code Walkthrough

Explicit Quantum Model





38th International Conference on VLSI Design & 24th International Conference on Embedded System



SILICON MEETS AI : Sustainable Innovation In Accelerated Computing, Secure Connectivity, And Intelligent Mobility
4th - 8th January 2025 | Bengaluru, India

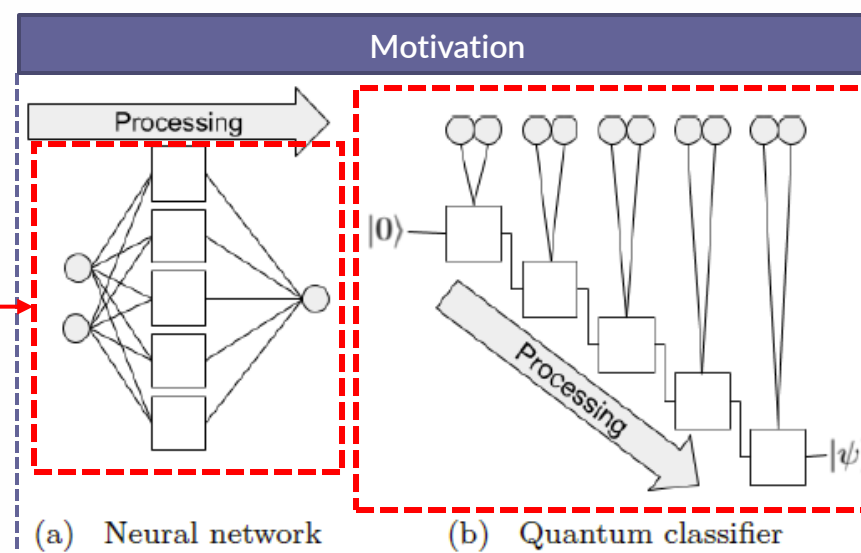
Section 4 – Data Reuploading Models.. Looking at the Details

Data Reuploading (DR) Model

A single layer deep neural network can be a universal classifier given an **infinite number of units**. This is due to the ability to **learn multiple functions of the same data**.

PROBLEM ! NO CLONING Theorem.

Problem Statement: The No-Cloning theorem makes it impossible to clone quantum data, and hence we cannot pass our features as inputs to different functions



Source: [1]

SOLUTION ! Data Reuploading

We can get around the no cloning theorem by **classically copying** our inputs and '**Reuploading**' them repeatedly back into the circuit.

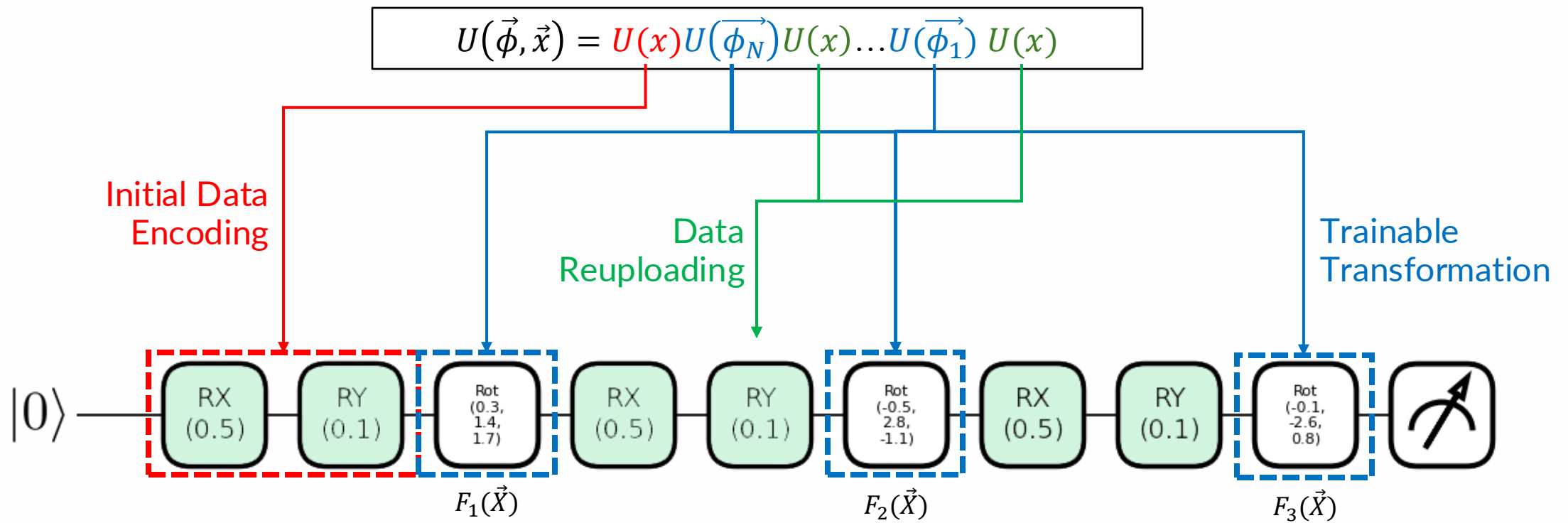
The resultant Unitary from our repeated encoding and processing:

$$U(\vec{\phi}, \vec{x}) = U(\vec{\phi}_N)U(x) \dots U(\vec{\phi}_1)U(x)$$

$\therefore$ The final state

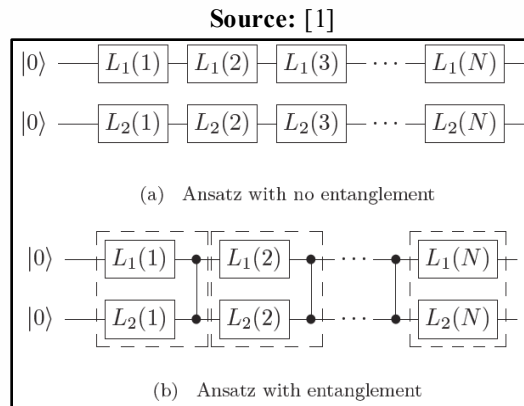
$$|\psi\rangle = U(\vec{\phi}, \vec{x})|0\rangle$$

Data Reuploading Circuit



Data Reuploading Variations

Multi-Qubit DR Models



Procedure: I. No Entanglement

We can trivially extend the DR framework to multiple qubits but thinking of *each qubit learning different features*.

Procedure: II. With Entanglement

When we extend the framework to multiple qubits, we can also take advantage of entanglement.

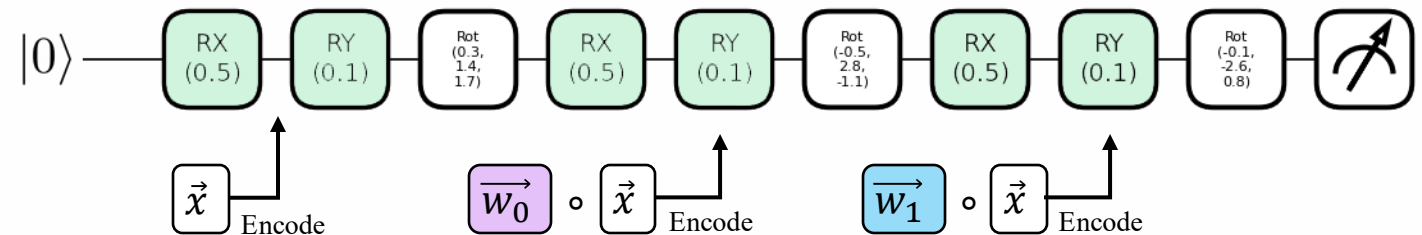
We can *create cyclic entanglement* between different qubits **similar to explicit models**.

IDEA ! Encode linear units.

Linear – Weighted DR Models

Weighted Sum DR Models

We can incorporate a *stronger idea from deep neural networks* into the DR framework – functions of weighted sum. A trivial but powerful modification of DR models is to swap the vanilla reuploading process with the **Dense Layer** style weighted features. This allows the model to learn richer functions from the data.



Generalized Form

$$L(i) = U(\vec{\theta}_i + \vec{w}_i \circ \vec{x})$$

The i^{th} Layer of our model

Resultant Unitary Operator

$$U(\vec{\phi}, \vec{x}, \vec{w}) = U(\vec{\phi}_N)U(w_N x) \dots U(\vec{\phi}_1)U(w_1 x)$$

The resultant unitary now learns even richer functions of x due to the weighted sums.

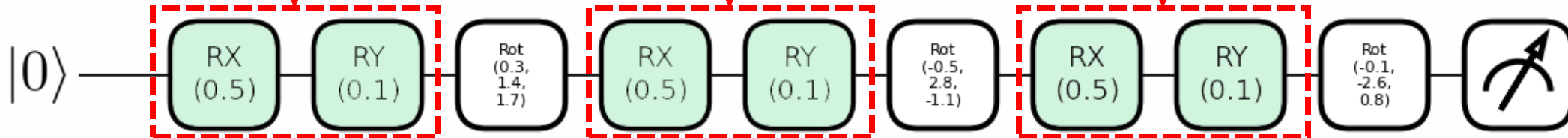
[1] A. Pérez-Salinas, A. Cervera-Lierta, E. Gil-Fuster, J.I. Latorre, Data re-uploading for a universal quantum classifier, *Quantum*, 4 (2020) 226.

DR Model in Code

Data Encoding

The DR Classifier **repeatedly** applies the data encoding layer, interleaving it with **variational compute** layers.

```
@qml.qnode(dev)
def DR_model(input, weights):
    for layer in range(weights.shape[0]):
        qml.RX(input[0], wires=0)
        qml.RY(input[1], wires=0)
        qml.Rot(*weights[layer], wires=0)
    return qml.expval(qml.PauliZ(0))
```

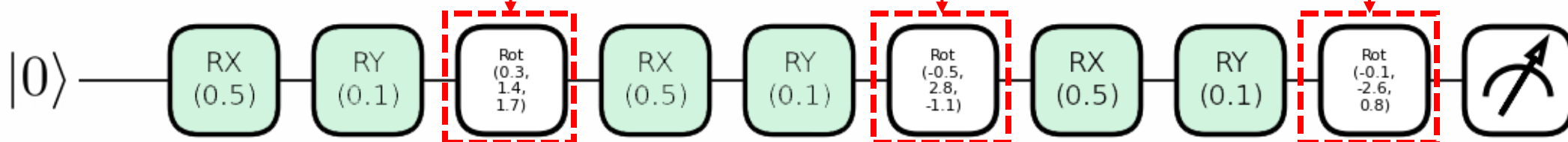


DR Model in Code

Variational Circuit

The variational circuits consists of parameterized $Rot(\theta_X, \theta_Y, \theta_Z)$ similar to explicit models, however, they interleaved with the data encoding

```
@qml.qnode(dev)
def DR_model(input, weights):
    for layer in range(weights.shape[0]):
        qml.RX(input[0], wires=0)
        qml.RY(input[1], wires=0)
        qml.Rot(*weights[layer], wires=0)
    return qml.expval(qml.PauliZ(0))
```

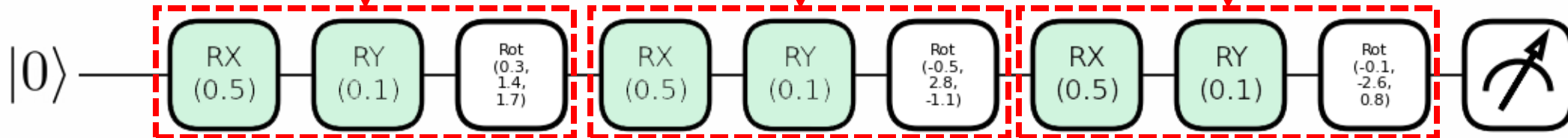


DR Model in Code

Layers in a DR Model

The layers of a DR model correspond to a **single set of data encoding and variational transformation.**

```
@qml.qnode(dev)
def DR_model(input, weights):
    for layer in range(weights.shape[0]):
        qml.RX(input[0], wires=0)
        qml.RY(input[1], wires=0)
        qml.Rot(*weights[layer], wires=0)
    return qml.expval(qml.PauliZ(0))
```

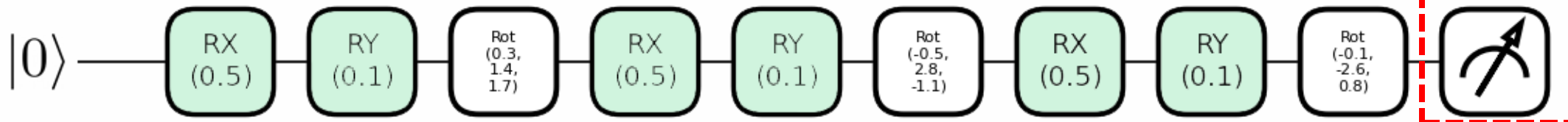


DR Model in Code

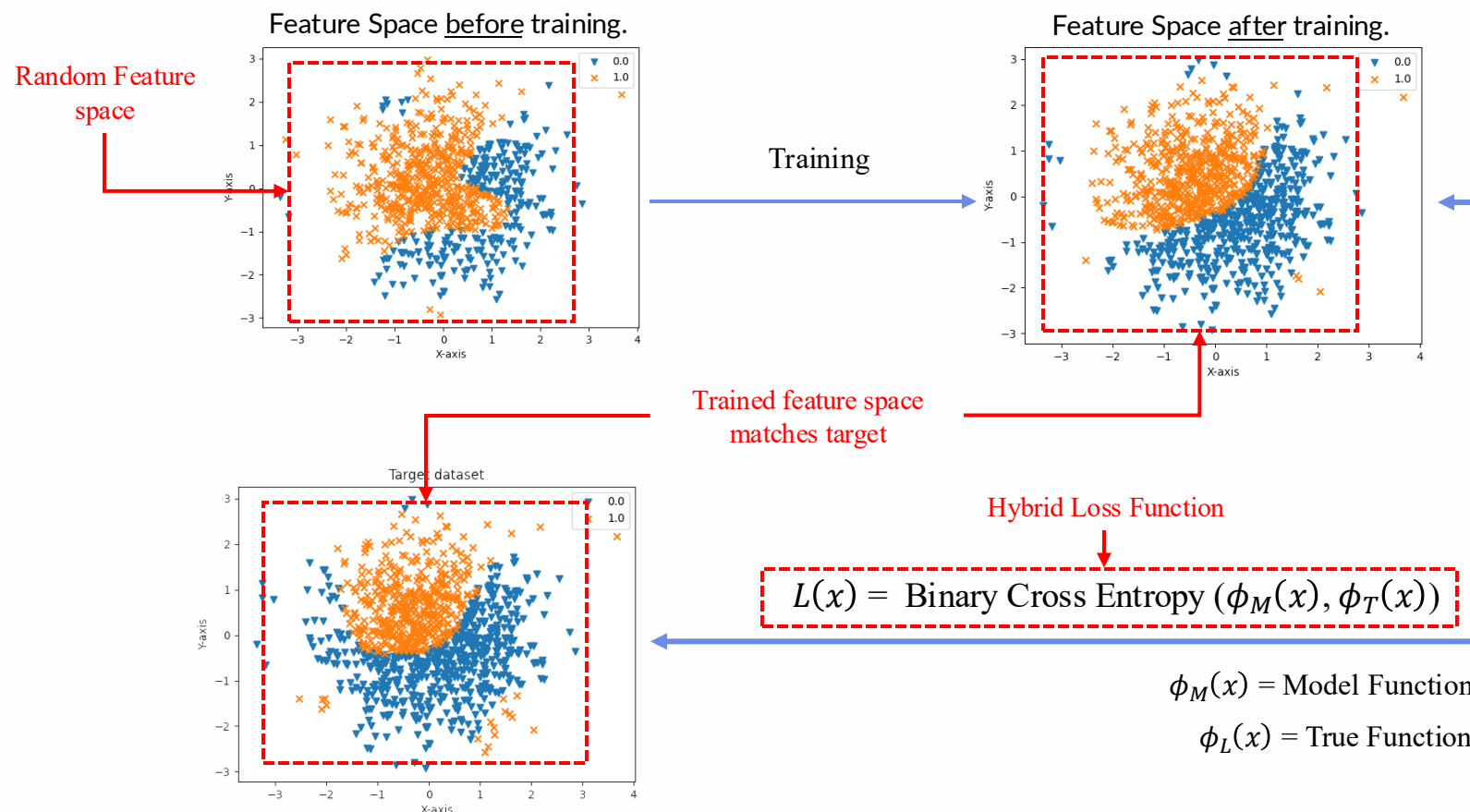
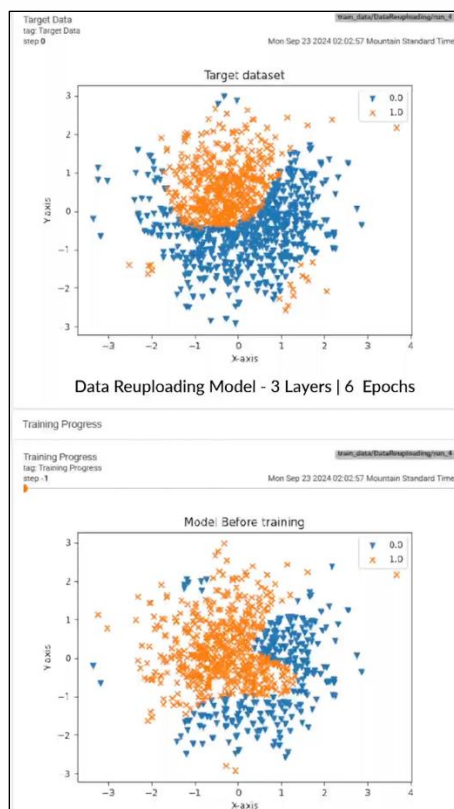
Measurement

We measure the expectation value w.r.t to the Pauli-Z operator to be used for our loss function.

```
@qml.qnode(dev)
def DR_model(input, weights):
    for layer in range(weights.shape[0]):
        qml.RX(input[0], wires=0)
        qml.RY(input[1], wires=0)
        qml.Rot(*weights[layer], wires=0)
    return qml.expval(qml.PauliZ(0))
```

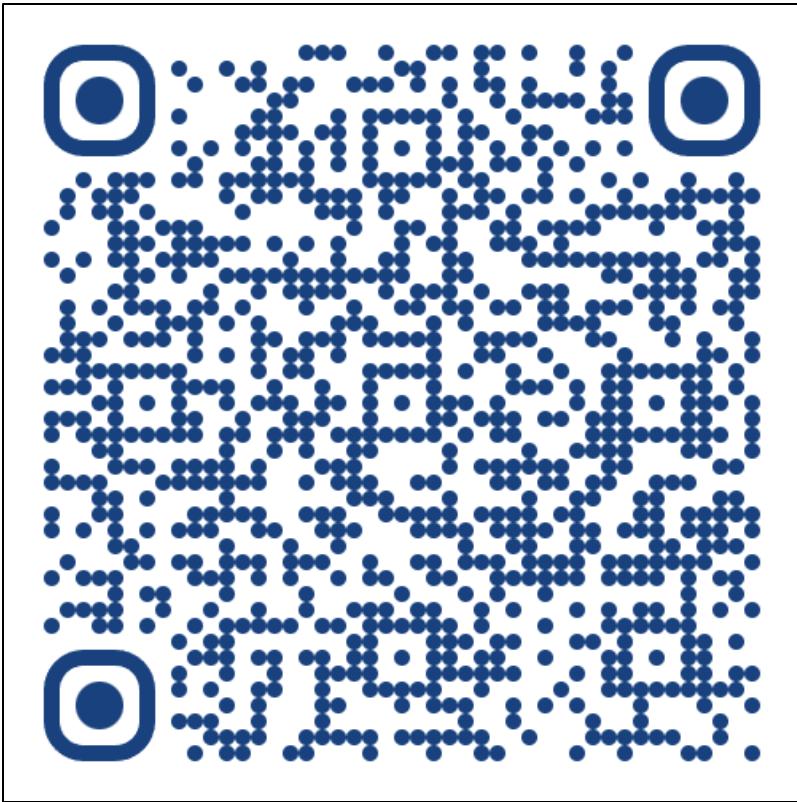


Training Evolution



Code Walkthrough

DR Quantum Model





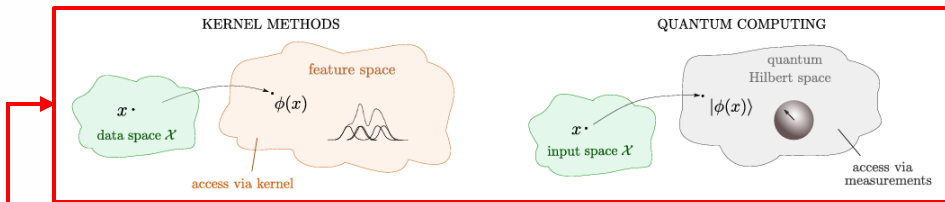
38th International Conference on VLSI Design & 24th International Conference on Embedded System



SILICON MEETS AI : Sustainable Innovation In Accelerated Computing, Secure Connectivity, And Intelligent Mobility
4th - 8th January 2025 | Bengaluru, India

Section 5 – Implicit Models.. Looking at the Details

Implicit Quantum Models



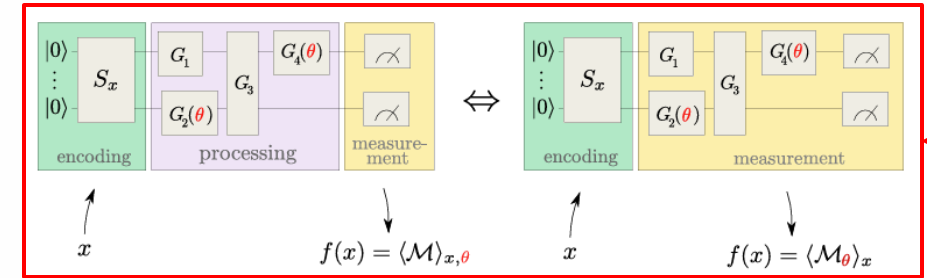
When compared against classical kernel methods we see a direct correlation between the 2 schemes

Quantum Models as Kernel Functions.

Statement: We can treat any unitary parameterized by our data as a feature map. Measuring against the adjoint of our U with different data creates a kernel function.

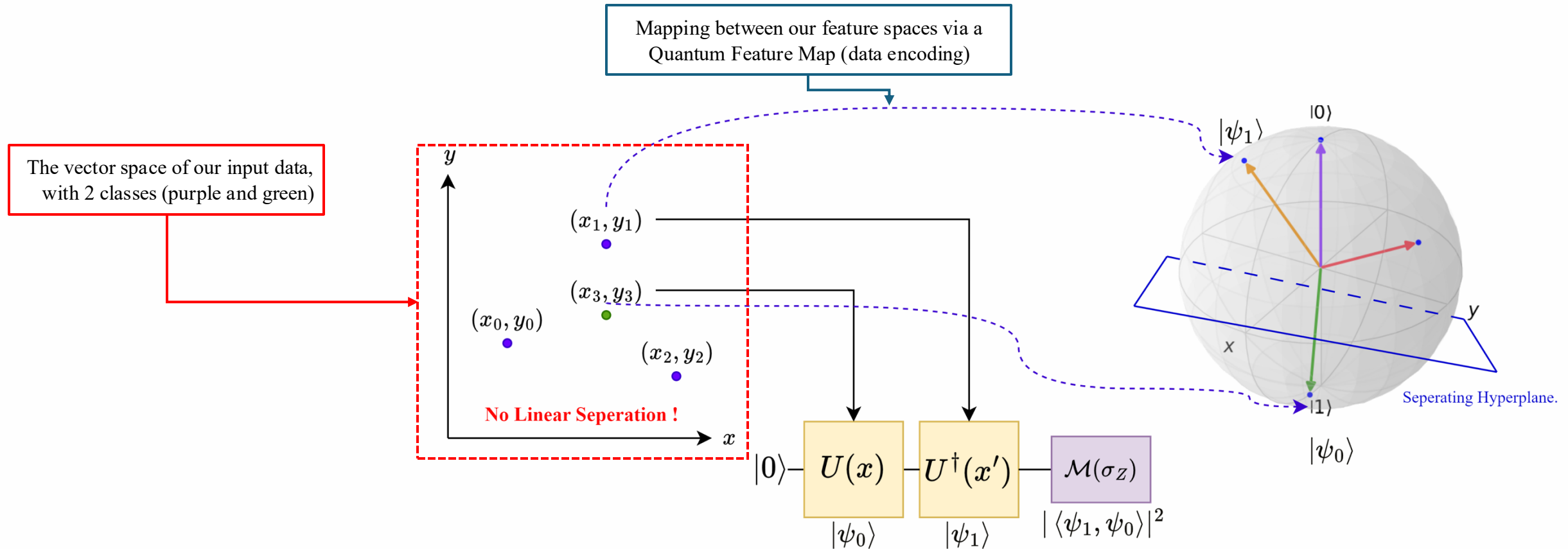
We can write down our implicit model, as a kernel function.

$$|0\rangle \xrightarrow[\psi_0]{U(x)} \xrightarrow[\psi_1]{U^\dagger(x')} \xrightarrow[\langle \psi_1, \psi_0 \rangle^2]{\mathcal{M}(\sigma_Z)} \equiv \mathcal{K}(x, x')$$

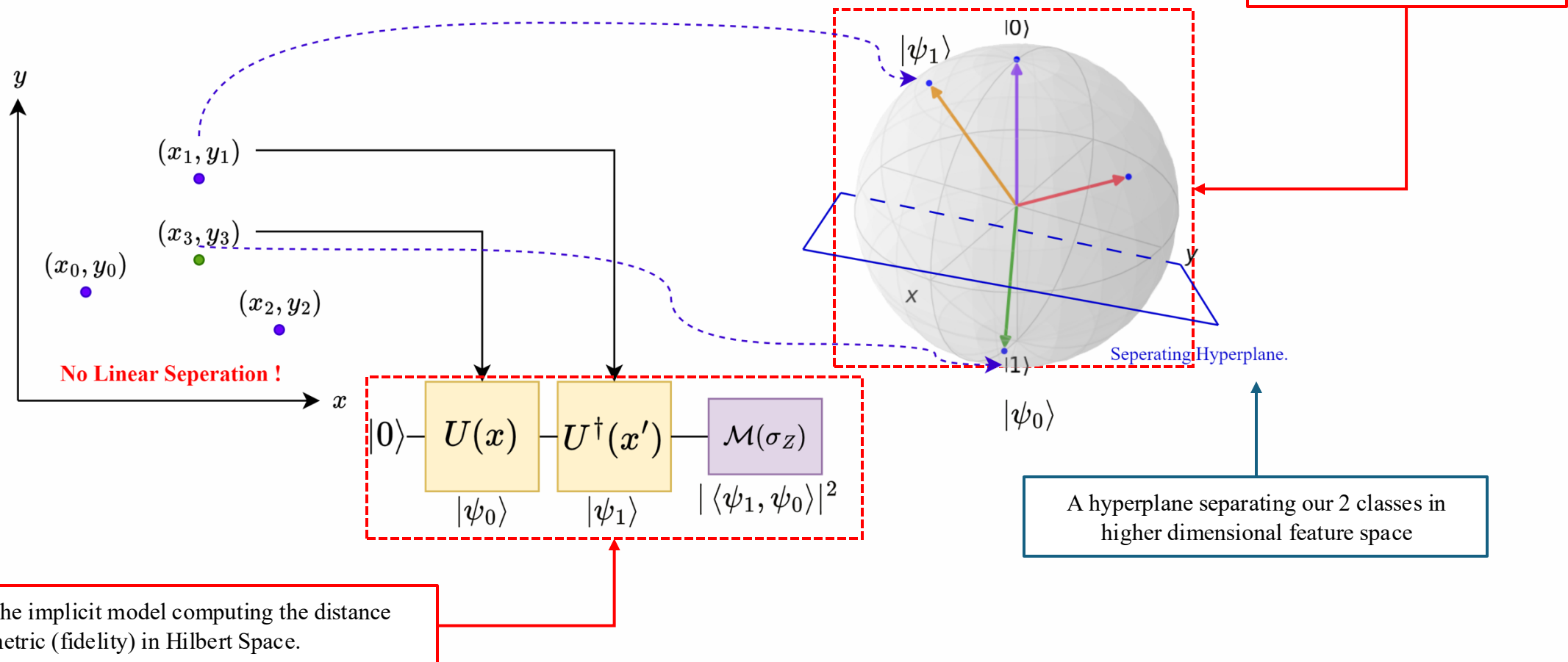


Under this interpretation we can re contextualize all explicit models as kernel methods where the variational model is trying to find the appropriate measurement basis (or separating hyperplane)

Visualizing the Kernel Process



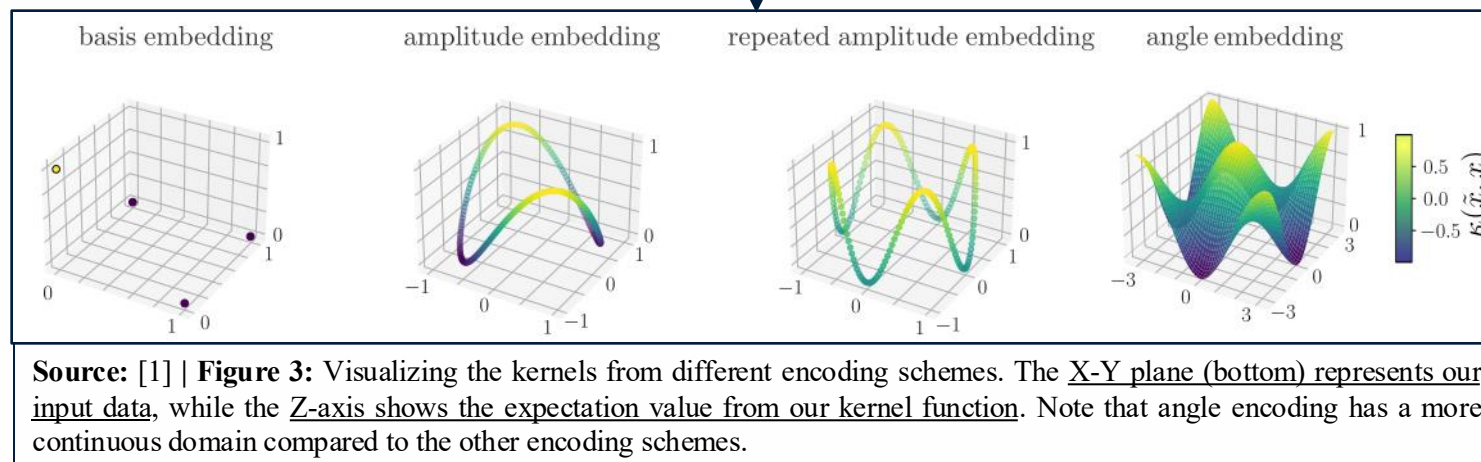
Visualizing the Kernel Process



Effect of Encoding on Kernel Map

Encoding	Kernel $k(x, x')$
Basis	$ \langle i_{x'} j_x \rangle ^2; i_x = \sum_{k=0}^{n-1} 2^k x_k$
Amplitude	$ x^\dagger x' ^2$
Rotational	$\prod_{k=1}^N \cos(x'_k - x_k) ^2$
Repeated Amplitude	$ x^\dagger x' ^{2r}$

Table 1: Kernel Functions created by different data encoding schemes.



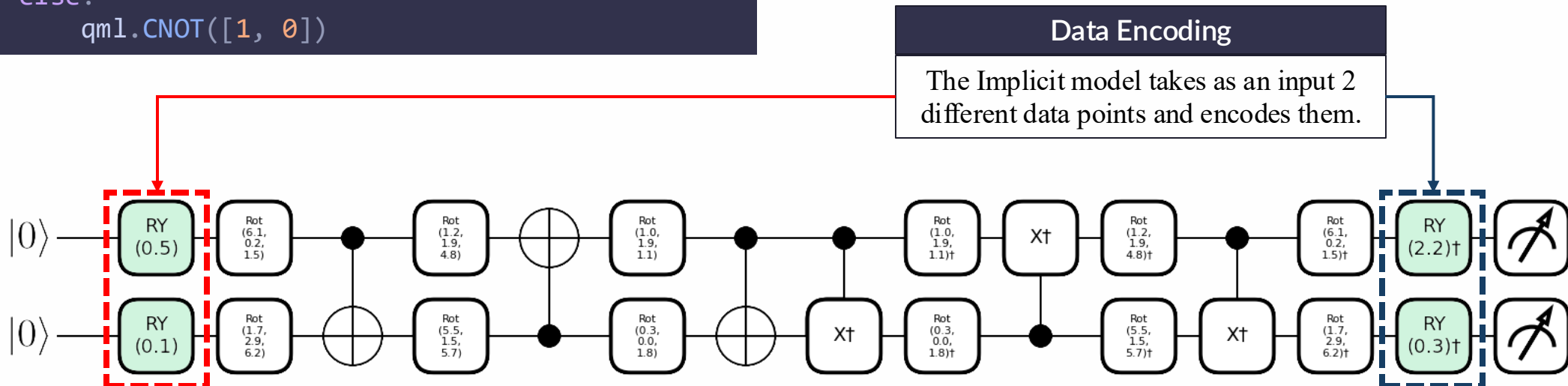
[1]. M. Schuld, Supervised quantum machine learning models are kernel methods, 2021.

Implicit Model in Code

```
def encoding(inp, weights):
    qml.RY(inp[0], wires=0)
    qml.RY(inp[1], wires=1)
    for layer in range(weights.shape[0]):
        for wire in range(2):
            qml.Rot(*weights[layer][wire], wires=wire)
        if(layer%2==0):
            qml.CNOT([0, 1])
        else:
            qml.CNOT([1, 0])
```

```
@qml.qnode(dev)
def Q_Kernel(x0, x1, weights):
    encoding(x0, weights)
    qml.adjoint(encoding)(x1, weights)
    return qml.probs(wires=range(n_qubits))

def implicit_model(x0, x1, weights):
    return Q_Kernel(x0, x1, weights)[0]
```

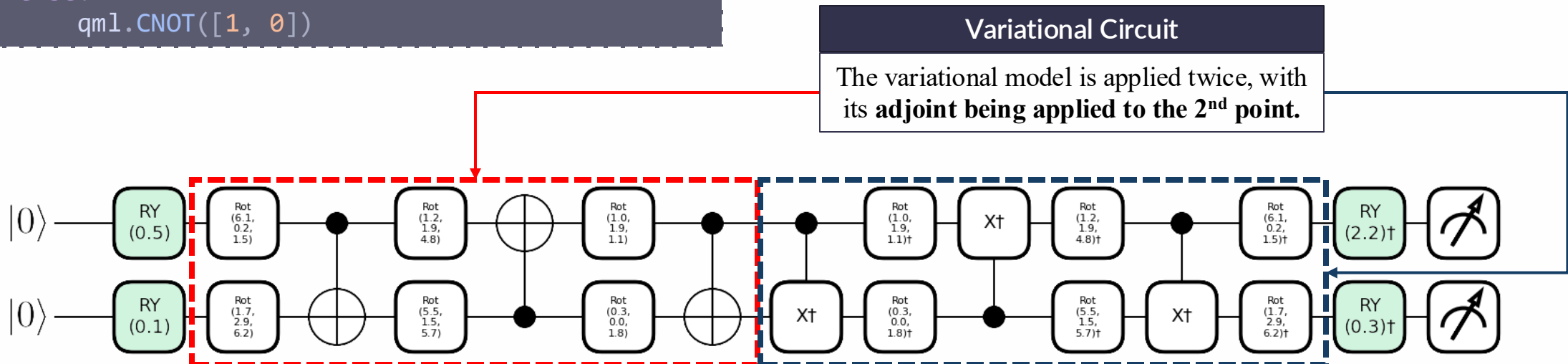


Implicit Model in Code

```
def encoding(inp, weights):
    qml.RY(inp[0], wires=0)
    qml.RY(inp[1], wires=1)
    for layer in range(weights.shape[0]):
        for wire in range(2):
            qml.Rot(*weights[layer][wire], wires=wire)
        if(layer%2==0):
            qml.CNOT([0, 1])
        else:
            qml.CNOT([1, 0])
```

```
@qml.qnode(dev)
def Q_Kernel(x0,x1, weights):
    encoding(x0,weights)
    qml.adjoint(encoding)(x1,weights)
    return qml.probs(wires=range(n_qubits))

def implicit_model(x0,x1, weights):
    return Q_Kernel(x0,x1, weights)[0]
```



Implicit Model in Code

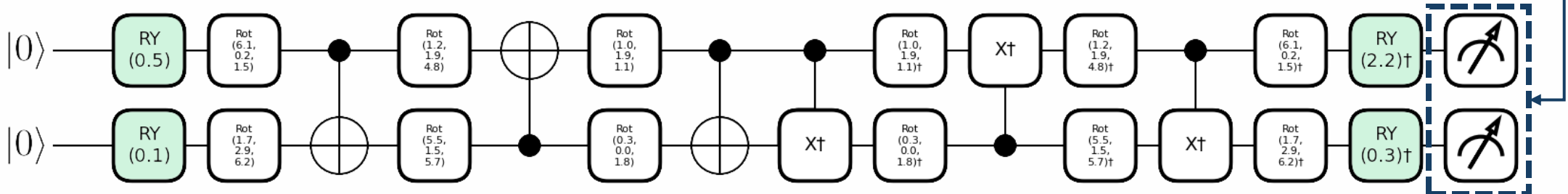
```
def encoding(inp, weights):
    qml.RY(inp[0], wires=0)
    qml.RY(inp[1], wires=1)
    for layer in range(weights.shape[0]):
        for wire in range(2):
            qml.Rot(*weights[layer][wire], wires=wire)
        if(layer%2==0):
            qml.CNOT([0, 1])
        else:
            qml.CNOT([1, 0])
```

```
@qml.qnode(dev)
def Q_Kernel(x0,x1, weights):
    encoding(x0,weights)
    qml.adjoint(encoding)(x1,weights)
    return qml.probs(wires=range(n_qubits))

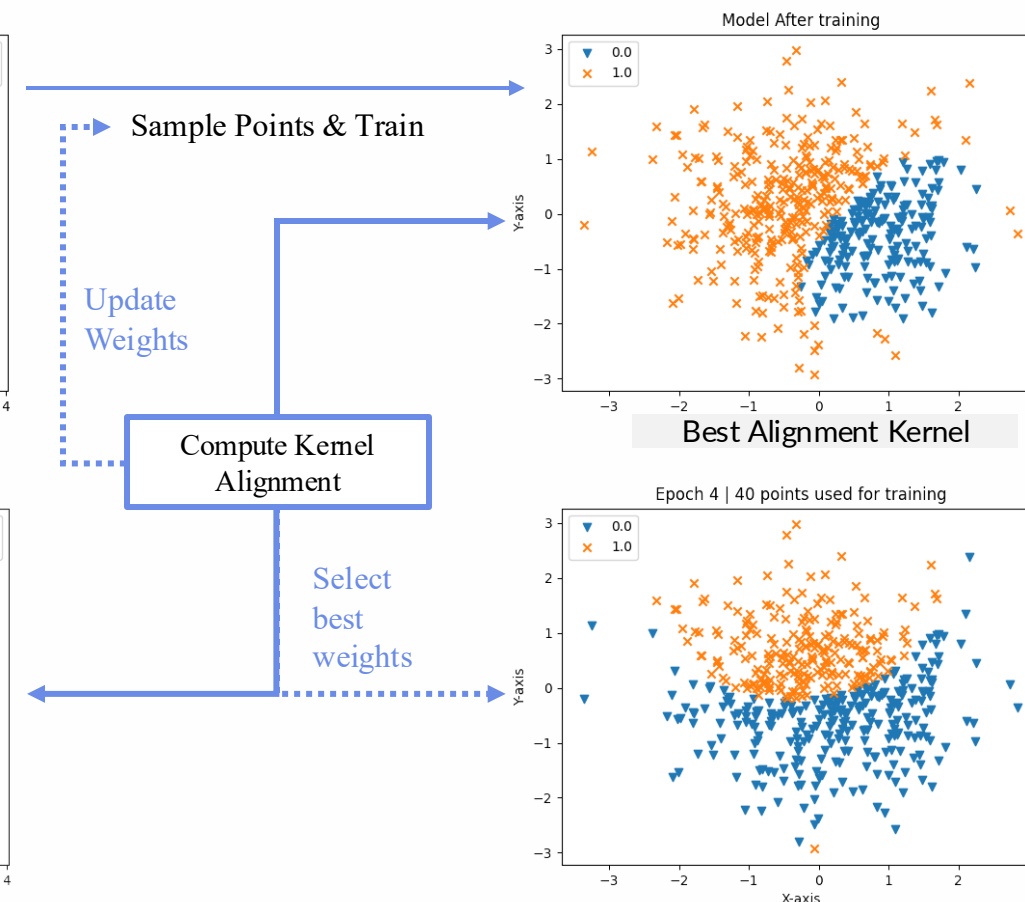
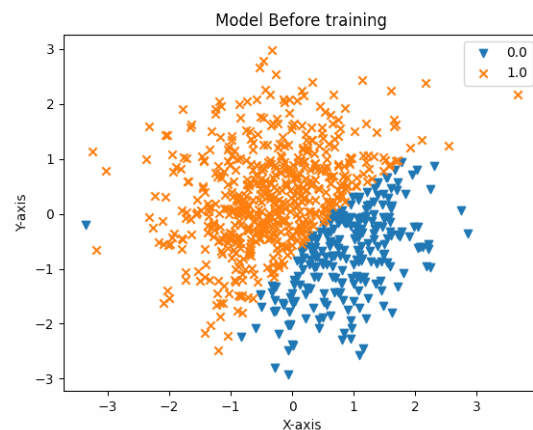
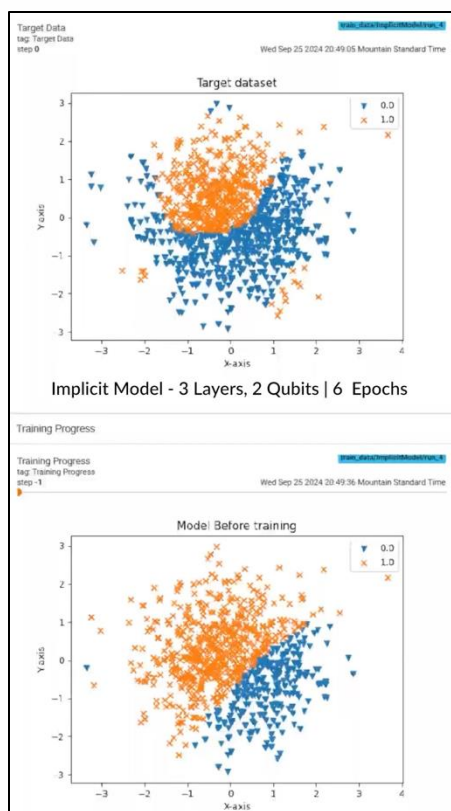
def implicit_model(x0,x1, weights):
    return Q_Kernel(x0,x1, weights)[0]
```

Measurement

Measurement in Implicit models computes the fidelity between 2 different states.



Training Evolution



Code Walkthrough

DR Quantum Model





38th International Conference on VLSI Design & 24th International Conference on Embedded System



SILICON MEETS AI : Sustainable Innovation In Accelerated Computing, Secure Connectivity, And Intelligent Mobility

4th - 8th January 2025 | Bengaluru, India